

MD5 应用场景, 什么特性, 底层实现

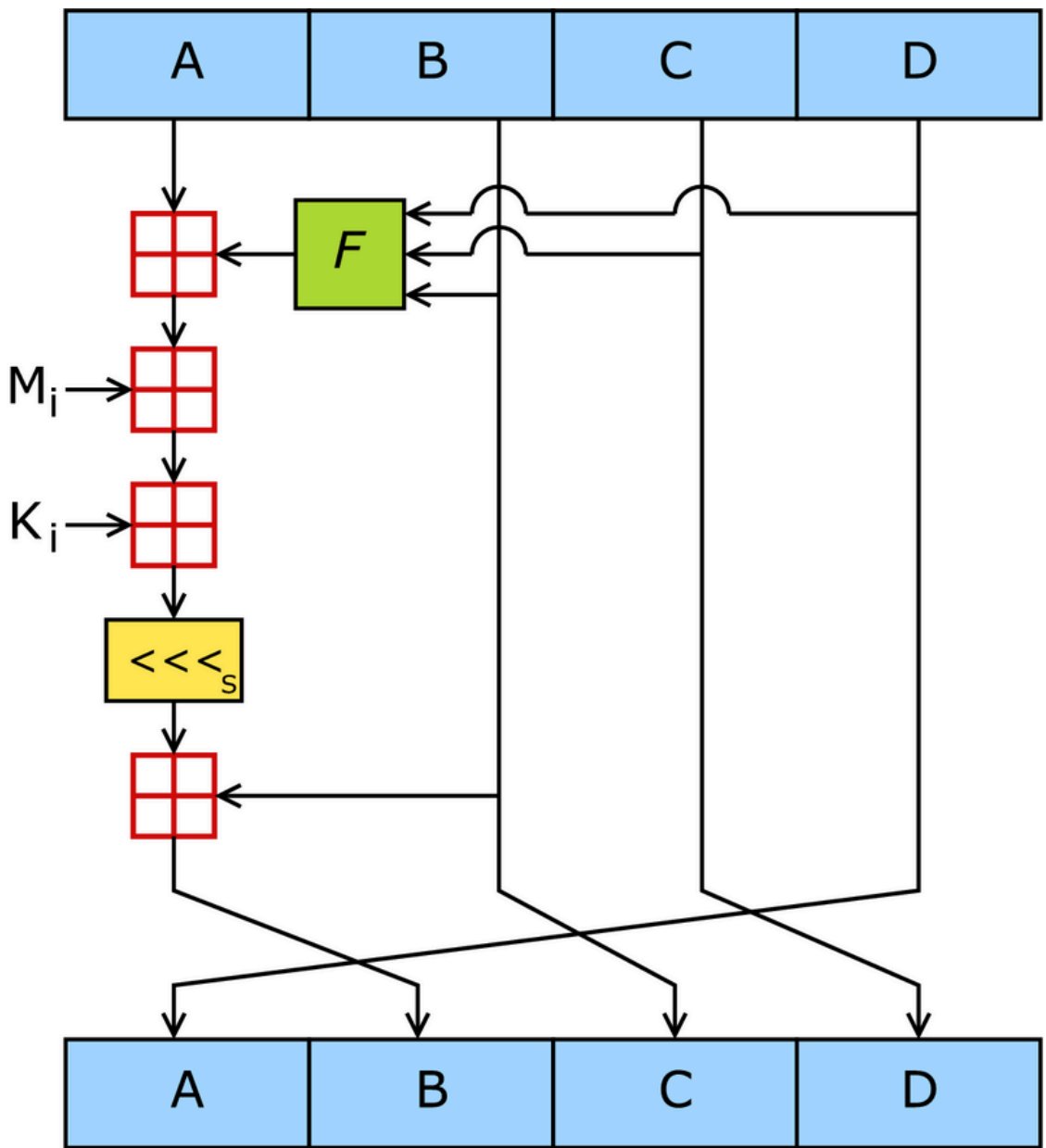
- 消息摘要算法第五版, 可生成 128 位哈希值
- 可用作校验和, 检验数据完整性, 防止意外发生
- 存在漏洞, 可被破解, 不适用于安全性认证, 建议改用其他算法例如SHA-2
- 曾用于SSL/TLS (现用SHA-1)、IPsec、SSH

底层实现

- MD5是输入不定长度信息，输出固定长度128-bits的算法。经过程序流程，生成四个32位数据，最后联合起来成为一个128-bits散列。基本方式为，求余、取余、调整长度、与链接变量进行循环运算。得出结果。

```
F(X,Y,Z) = (X^Y) v (~X^Z)
G(X,Y,Z) = (X^Z) v (Y^~Z)
H(X,Y,Z) = X^Y^Z
I(X,Y,Z) = Y^ (X v ~Z)
```

⊕, ∧, ∨, ¬ 是 XOR, AND, OR , NOT 的符号



一个MD5运算- 由类似的64次循环构成，分成4组16次

F 一个非线性函数；一个函数运算一次。 M_i 表示一个 32-bits 的输入数据， K_i 表示一个 32-bits 常数，用来完成每次不同的计算

项目中MD5怎么做的，会有风险吗，加盐就安全了吗

项目中MD5主要在客户端处理

用于计算上传的文件每一个二进制字节，哈希值作为服务器存储的文件名，同时存储在数据库用于查询
其次，用于用户注册登录时密码的加密传输，数据库同样存储

有风险，且有漏洞，服务器同样在文件上传完成后应当计算MD5进行校验，同时处理密码时应当为每位用户动态加盐

原因是可被彩虹表暴力破解，若静态加盐，可被猜测静态加盐规律

可以考虑采用其他加密算法，例如刚才提到的 [SHA-2](#) (Secure Hash Algorithm) 亦或者是 [CRC](#)]

(<https://zh.wikipedia.org/wiki/%E5%BE%AA%E7%92%B0%E5%86%97%E9%A4%98%E6%A0%A1%E9%A9%97> 循环冗余校验)

其他信息摘要 [SHA](#)，[CRC](#) 了解吗

- [SHA](#) 家族有很多安全散列算法，比如SHA-1系列，SHA-2系列，其中SHA-1用于TLS/SSL安全性认证，但也存在碰撞漏洞，而SHA-2系列，比如SHA-256目前广泛使用，在GitHub等开源网站下载时很常见，认为是较安全的
- [CRC](#) 循环冗余校验，非加密，检验数据是否错误，不可靠检验完整性，长度可变，主要用于数据传输中的错误，例如用于网络传输中，不适合密码，因为易于伪造

几十亿文件数据，查找MD5，怎么优化

1. 以MD5作为前缀索引，在进行以MD5模糊查询时能加快查询速度
2. 水平分表，按 MD5 值的某些属性（如前几位）决定数据存入哪个表
3. 使用 [redis](#) 缓存，缓存频繁查询的 MD5 数据
4. 使用[布隆过滤器](#)快速排除不可能存在的数据

WebSocket了解吗

- 一种[网络传输协议](#)，可在单个[TCP](#)连接上进行[全双工](#)通信，位于[OSI模型](#)的[应用层](#)
- 使得客户端和服务端之间的数据交换变得更加简单，允许服务端主动向客户端推送数据。在WebSocket API中，浏览器和服务器只需要完成一次握手，两者之间就可以建立持久性的连接，并进行双向数据传输
- WebSocket是一种与[HTTP](#)不同的协议。两者都位于[OSI模型](#)的[应用层](#)，并且都依赖于[传输层](#)的TCP协议。虽然它们不同，但是RFC 6455中规定：[it is designed to work over HTTP ports 80 and 443 as well as to support HTTP proxies and intermediaries](#) (WebSocket通过HTTP端口80和443进行工作，并支持HTTP代理和中介)，从而使其与HTTP协议兼容。为了实现兼容性，WebSocket握手使用HTTP Upgrade头1从HTTP协议更改为WebSocket协议。

TCP和HTTP的keep-alive的区别

TCP如何实现保活

- HTTP Keep-Alive 由[应用层（用户态）](#) 实现的，是HTTP长连接；
- 避免短连接频繁建立连接和释放连接的开销；
- TCP 保活为TCP机制，心跳为用户在应用层定义的。保活AKA存活包
- 存活包，为可选特性，且默认关闭

- 存活包内没有数据
- 在以太网网络中，存活包的大小为最小长度的几帧64字节

协议中，还有三个与存活包相关的参数：

- **存活时长**Keepalive time：即空闲时，两次传输存活包的持续时间。TCP存活包时长可手动配置，默认不少于2个小时。
- **存活间隔**Keepalive interval：即未收到上个存活包时，两次连续传输存活包的时间间隔。
- **存活重试次数**Keepalive retry：即在判断远程主机不可用前的发送存活包次数。当两个主机透过TCP/IP协议相连时，TCP存活包可用于判断连接是否可用，并按需中断。

多数支持TCP协议的主机也同时支持TCP存活包

1. 每个主机按一定周期向其他主机发送TCP包来请求回应
2. 若发送主机未收到特定主机的回应（ACK），则将从发送主机一侧中断连接
3. 若其他主机在连接关闭后发送TCP存活包，关闭连接的一方将发送RST包来表明旧连接已不可用
4. 其他主机将关闭它一侧的连接以新建连接

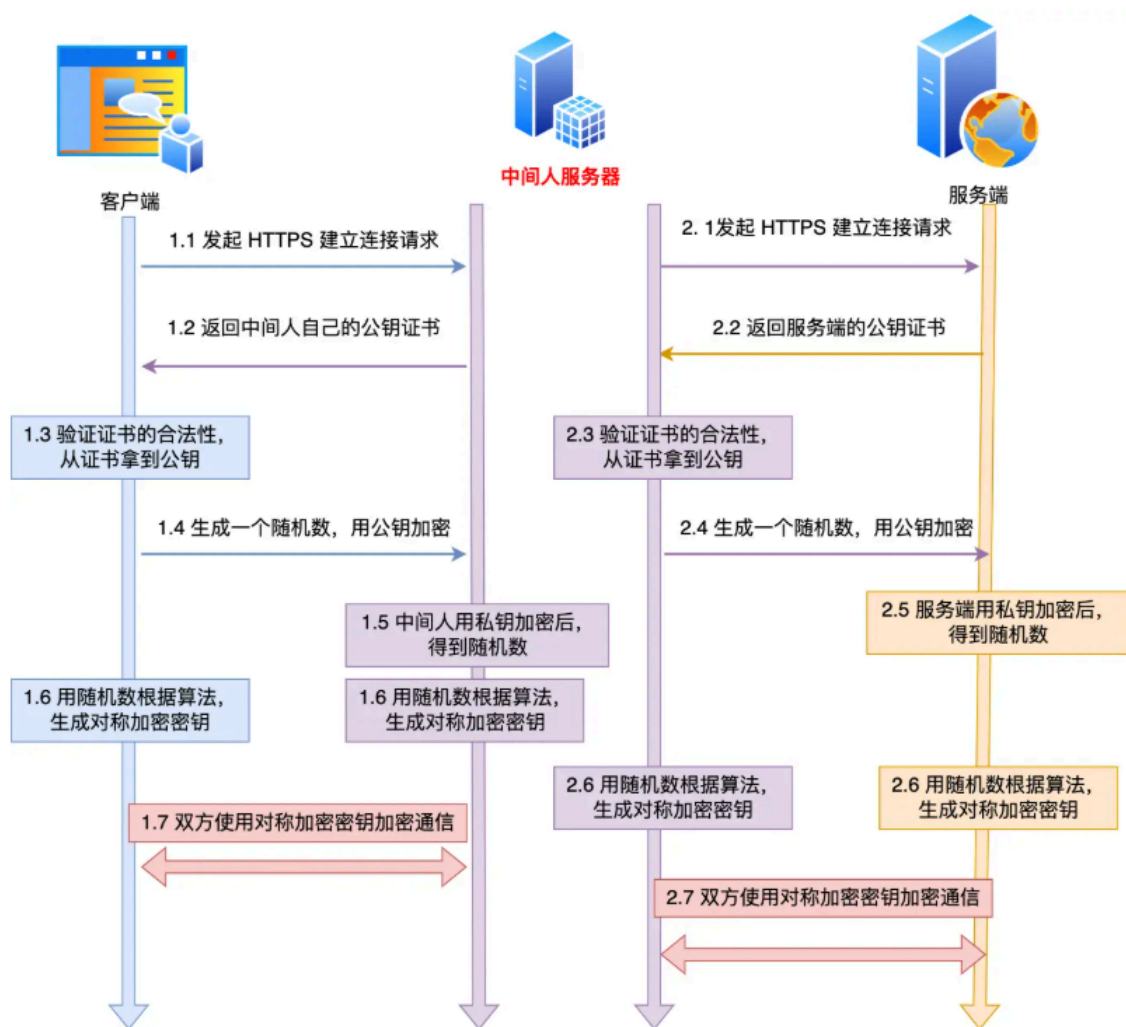
- 空闲的TCP连接通常会隔每45秒或60秒发送一次存活包
- 在未连续收到三次ACK包时，连接将中断。此行为因主机而异，如默认情况下的Windows主机将在7200000ms（2小时）后发送首个存活包，随后再以1000ms的间隔发送5个存活包
- 若任意存活包未收到回应，连接将被中断

HTTP

- HTTP使用在“Connection”头部信息中使用关键字“Keep-Alive”来指示连接应保持开启以接收之后的信息，这是HTTP 1.1中的默认情形，而HTTP 1.0默认将为每对请求/回复对创建新连接

HTTPS一定是安全的吗,有什么风险

- 客户端通过浏览器向服务端发起 HTTPS 请求时，被「假基站」转发到了一个「中间人服务器」，于是客户端是和「中间人服务器」完成了 TLS 握手，然后这个「中间人服务器」再与真正的服务端完成 TLS 握手



❗ 前提, 前提是用户点击接受了中间人服务器的证书

中间人服务器与客户端在 TLS 握手过程中, 实际上发送了自己伪造的证书给浏览器, 而这个伪造的证书是能被浏览器 (客户端) 识别出是非法的, 于是就会提醒用户该证书存在问题

线程池活跃线程数达到 MAX, 线程全部繁忙, 任务队列已满, 怎么处理

- 加缓存

在消费者-生产者模型(HS/HA半同步/半异步模式)下的线程池中, 如果活跃线程达到最大且全部繁忙, 同时任务队列已满, 可以考虑以下几种策略:

1. 拒绝策略:

- **抛出异常**: 直接抛出一个异常, 通知调用者任务无法被接受
- **调用者运行**: 让调用者在当前线程中执行任务, 而不是提交到线程池
- **丢弃任务**: 丢弃最旧的任务或最常见的任务, 通常用于低优先级的任务

2. 阻塞策略: 阻塞队列: 使用阻塞队列, 提交任务时如果队列满则阻塞当前线程, 直到有空间可用

3. 扩展线程池: 动态扩容: 根据负载情况动态增加线程池的最大线程数 (需谨慎使用, 以避免资源耗尽)

4. 等待与重试: 等待重试: 让任务在一定时间后重试提交, 增加成功的几率

5. 监控与警报: 监控系统性能: 定期监控线程池的状态, 设置警报以便及时响应

HS/HA半同步/半异步模式、L/F领导者与跟随者模式

- 半同步/半异步模式又称为生产者消费者模式，是比较常见的实现方式，比较简单。分为同步层、队列层、异步层三层。同步层的主线程处理工作任务并存入工作队列，工作线程从工作队列取出任务进行处理，如果工作队列为空，则取不到任务的工作线程进入挂起状态。由于线程间有数据通信，因此不适用于大数据量交换的场合。
- 领导者跟随者模式，在线程池中的线程可处在3种状态之一：领导者leader、追随者follower或工作者processor。任何时刻线程池只有一个领导者线程。事件到达时，领导者线程负责消息分离，并从处于追随者线程中选出一个来当继任领导者，然后将自身设置为工作者状态去处置该事件。处理完毕后工作者线程将自身的状态置为追随者。这一模式实现复杂，但避免了线程间交换任务数据，提高了CPU cache相似性。在ACE(Adaptive Communication Environment)中，提供了领导者跟随者模式实现。

Java ThreadPoolExecutor 任务拒绝策略

拒绝策略：

```
提交任务 → 核心线程是否已满？
├─ 未满 → 创建核心线程执行
└─ 已满 → 任务入队
    ├─ 队列未满 → 等待执行
    └─ 队列已满 → 创建非核心线程
        ├─ 未达最大线程数 → 执行任务
        └─ 已达最大线程数 → 执行拒绝策略
```

候选者：ThreadPoolExecutor在构造的时候有几个重要的参数，分别是：

- corePoolSize（核心线程数量）
- maximumPoolSize（最大线程数量）
- keepAliveTime（线程空余时间）
- workQueue（阻塞队列）
- handler（任务拒绝策略）

候选者：这几个参数应该很好理解哈，我就说下任务提交的流程，分别对应着几个参数的作用吧

1. 首先会判断运行线程数是否小于corePoolSize，如果小于，则直接创建新的线程执行任务
2. 如果大于corePoolSize，判断workQueue阻塞队列是否已满，如果还没满，则将任务放到阻塞队列中
3. 如果workQueue阻塞队列已经满了，则判断当前线程数是否大于maximumPoolSize，如果没大于则创建新的线程执行任务
4. 如果大于maximumPoolSize，则执行任务拒绝策略（具体就是你自己实现的handler）

候选者：这里有个点需要注意下，就是workQueue阻塞队列满了，但当前线程数小于maximumPoolSize，这时候会创建新的线程执行任务

四种预置的拒绝策略：

- CallerRunsPolicy，使用线程池的调用者所在的线程去执行被拒绝的任务，除非线程池被停止或者线程池的任务队列已有空缺
- AbortPolicy，直接抛出一个任务被线程池拒绝的异常
- DiscardPolicy，不做任何处理，静默拒绝提交的任务
- DiscardOldestPolicy，抛弃最老的任务，然后执行该任务
- 自定义拒绝策略，通过实现接口可以自定义任务拒绝策略